

# Hangman Lab – LLM Usage Report

Oct 9, 2026 · @Mathin

## Overview

I completed all four tasks of Scenario 16 (Hangman Challenge) with Claude as a coding assistant, then tested the result by hand and caught a scoring question that led to one more change.

| Item               | Detail                                                    |
|--------------------|-----------------------------------------------------------|
| Student            | Mathin M (PES1UG24AM377), B.Tech CSE (AI & ML), Section G |
| Scenario           | 16 — Hangman Challenge ( 16_hangman )                     |
| LLM                | Claude (claude.ai), single chat session                   |
| Files changed      | game.py (modified), test_game.py (new)                    |
| Files untouched    | main.py, words.py, stats.py, requirements.txt, README.md  |
| Dependencies added | None (standard library only)                              |
| Persistence added  | None (all state in memory)                                |

Complete LLM chat history: <https://claude.ai/share/34452c86-388a-4344-9fe1-8406667225b7>. This report summarises the work done in it.

## Session part 1 — Inspect, reproduce, fix

**My prompt:** I uploaded the project zip ( 16\_hangman.zip ) and asked for help completing the lab challenge.

### 1. Inspection before changes

Claude read every file ( README.md , main.py , game.py , words.py , stats.py , requirements.txt ) and the git state. The working-tree diffs against the repo were line-ending (CRLF) changes only, so the code matched the original commit.

Defects found in the original game.py :

1. `guess()` checks `self.guessed` but never `self.wrong`, so a repeated wrong letter costs another life (Task 1 bug).
2. `stats.py` (`SessionStats`) is supplied but never used, so rounds played, wins and best streak are not tracked.
3. `use_hint()` does `max(0, score - 1)`, so a hint is free when the score is 0 — inconsistent scoring.
4. `/quit` exits without printing a final score; there is no session summary.
5. Any single Unicode letter (e.g. `é`) passes `isalpha()`, and invalid category or y/n input is not handled well.

## 2. Reproducing the Task 1 bug

The original code was checked out from git and run with category `technology`, guessing `z` three times:

```
Lives: 6 Score: 0 Streak: 0
Lives: 5 Score: 0 Streak: 0
Lives: 4 Score: 0 Streak: 0
Lives: 3 Score: 0 Streak: 0
```

One wrong letter cost three lives — the defect is confirmed.

## 3. Changes made, by task

**Task 1 — Guess-state correctness.** `Round.guess()` now checks both the correct and wrong sets first. A repeat prints “You already tried 'z' ... No life lost.” or “You already found 'p'. Nothing changed.” Each distinct letter changes state exactly once.

**Task 2 — Session model.** Round-only state (secret, correct/wrong letters, lives, hint flag, difficulty) moved into a new `Round` class; each round creates a fresh `Round`, so nothing can leak. `HangmanGame` keeps session state (score, streak, `SessionStats`, chosen category/difficulty). `finish_round()` updates streak, score and `stats.record()`. A session summary (rounds, wins, win rate, best streak, final score) prints on every exit path, including `q`, `/quit`, `n` and Ctrl-C/EOF.

**Task 3 — Difficulty and scoring.**

| Difficulty | Lives | Multiplier | Hint cost |
|------------|-------|------------|-----------|
| easy       | 8     | ×1         | 1         |
| normal     | 6     | ×2         | 2         |
| hard       | 4     | ×3         | 3         |

Win points =  $(5 + \text{streak}) \times \text{multiplier} - \text{hint cost}$  (never below 0); a loss earns 0. The hint cost is taken from the round's reward, so it is the same whatever the current score. Pressing Enter keeps the previous category and difficulty.

**Task 4 — Input and feedback.** Letters must be a single `a-z`. Empty input, multiple characters, digits, symbols, `é` and unknown `/commands` each print one message and change nothing. Category and difficulty accept a name or a number and re-prompt on invalid input. The y/n prompt re-asks on junk. A second `/hint` re-shows the hint at no extra cost.

## 4. Automated tests

Claude added `test_game.py` (standard-library `unittest`, 14 tests). The first run had one error: the scripted-input helper raised `StopIteration` instead of `EOFError` when inputs ran out. That was a test-harness bug, not a game bug; after fixing the helper, all 14 tests passed. A piped sample session confirmed the repeated `z`, invalid `abc`, double `/hint` and `/quit` behaviour.

## Session part 2 — Manual testing and a scoring query

**My prompt:** I ran the updated game on my own machine and pasted the full terminal output back. I tested two sessions:

1. **easy / technology:** guessed `z` twice — lives went  $8 \rightarrow 7$  and the repeat said “No life lost”. Then `/quit` mid-round showed the word and a summary with 0 rounds played.
2. **hard / science (word: enzyme):** used `/hint` twice (charged once), guessed e, n, z, y, m and solved it. The game showed **+15 points**, final score 15.

I questioned the result: I expected 13 and thought the  $-3$  hint penalty was not being applied. I asked for only the updated file.

**Analysis:** Claude worked through the formula. On hard, a win is  $(5 + \text{streak } 1) \times 3 = 18$ , and the hint takes 3, giving 15. The scoring was correct; the problem was that the win message gave no breakdown, so the penalty was invisible to the player.

**Fix:** only the win message in `play_round()` changed, to show the calculation:

Solved: enzyme +15 points [(5 + streak 1) x 3 = 18, hint -3]

Scoring logic was not changed. All 14 tests still passed, and the enzyme/hard/hint case was replayed to confirm the new message.

## Testing summary

Every testing area the README requires is covered by an automated test, a manual run, or both; all passed.

| README area              | How tested                                                              | Result    |
|--------------------------|-------------------------------------------------------------------------|-----------|
| Repeated incorrect guess | Unit test ( z ×3 → 1 life lost); manual run on easy                     | Pass      |
| Repeated correct guess   | Unit test ( p twice → counted once)                                     | Pass      |
| Letter appearing twice   | Unit test ( a in galaxy); manual ( e in enzyme)                         | Pass      |
| Hints                    | Unit tests (charged once, cost at score 0); manual double /hint on hard | Pass      |
| Category changes         | Scripted session (invalid name, number, Enter to keep)                  | Pass      |
| Multiple rounds          | Unit test: new round resets round state, keeps session state            | Pass      |
| Streak resets            | Unit test: win, win, loss, win → best streak 2, current 1               | Pass      |
| Difficulty changes       | Unit tests: lives per level, no leak hard → easy; manual easy and hard  | Pass      |
| Invalid input            | Unit test: "", ab, 1, ?, é, /foo change nothing; bad y/n re-prompts     | Pass      |
| Quitting                 | Unit tests for /quit and EOF; manual /quit and q show the summary       | Pass      |
| Original bug reproduced  | Original commit run: lives 6 → 3 on z ×3                                | Confirmed |

Run the tests with `python -m unittest -v test_game`.

## LLM usage and verification

Claude inspected the code, explained the defects, wrote the changes and the unit tests. I ran the game myself, checked the behaviour against the README, and challenged an output I did not understand.

- **Inspected first:** the existing files and git state were read before any change, and the Task 1 bug was reproduced on the original code.
- **Kept the project:** only `game.py` was modified; the structure, `words.py` and the supplied `stats.py` were reused, not replaced.
- **Tested by me:** two manual sessions (easy and hard) covering repeated guesses, hints, solving a word and quitting.
- **Asked for an explanation:** I queried the 15-point score; the explanation showed the formula was right and led to clearer feedback.